

Introducción a Pandas

Clase 13

¿Qué es Pandas?

- Pandas es una **biblioteca de análisis de datos** de código abierto escrita en Python.
- Proporciona **estructuras de datos** de alto rendimiento y fáciles de usar, así como herramientas para manipular y analizar datos.
- Pandas es ampliamente utilizado en el campo de la ciencia de datos y se ha convertido en una herramienta esencial para cualquier persona que trabaje con datos en Python.

Estructuras de datos en Pandas

- Pandas proporciona dos estructuras de datos principales: Series y DataFrame.
- Una **Series** es una estructura de datos unidimensional que puede contener cualquier tipo de datos, como números, cadenas de texto, fechas, etc.
- Un **DataFrame** es una estructura de datos bidimensional similar a una tabla que contiene múltiples columnas y filas. Se puede pensar en un DataFrame como una hoja de cálculo o una tabla de base de datos.
- Tanto las Series como los DataFrame ofrecen numerosas funcionalidades y métodos para manipular, limpiar, analizar y visualizar datos de manera eficiente y sencilla.

Funcionalidades

- **Importación y exportación de datos:** Pandas permite leer y escribir datos desde y hacia una variedad de formatos, como CSV, Excel, bases de datos, archivos JSON, entre otros.
- **Manipulación y limpieza de datos:** Pandas proporciona métodos para filtrar, ordenar y transformar los datos. Además, ofrece herramientas para tratar datos faltantes, eliminar duplicados y aplicar operaciones en columnas o filas.
- **Operaciones estadísticas:** Pandas facilita el cálculo de estadísticas descriptivas, como la media, la mediana, el mínimo, el máximo y la desviación estándar. También permite realizar operaciones de agregación y resumen de datos.
- **Agrupación y agregación:** Con Pandas, puedes agrupar los datos según una o más columnas y aplicar operaciones de agregación, como sumas, promedios o conteos, en grupos específicos.
- **Fusión y concatenación de datos:** Pandas permite combinar datos de diferentes fuentes a través de operaciones de fusión y concatenación. Esto es útil cuando se desea combinar múltiples conjuntos de datos en uno solo.
- **Visualización de datos:** Pandas se integra con bibliotecas de visualización, como Matplotlib y Seaborn, para generar gráficos y visualizaciones de datos de manera sencilla.

pd.Series

```
import pandas as pd
```

```
top_3 = pd.Series([13.42, 11.56, 10.80], index=["Python", "C", "C++"])
```

```
print(top_3)
```

```
print(type(top_3))
```

```
print(top_3["C++"])
```

pd.DataFrame

```
datos = {  
    'tenista': ['Novak Djokovic', 'Rafael Nadal', 'Roger Federer', 'Ivan Lendl', 'Pete Sampras', 'John McEnroe',  
    'Bjorn Borg'],  
    'pais': ['Serbia', 'España', 'Suiza', 'República Checa', 'Estados Unidos', 'Estados Unidos', 'Suecia'],  
    'gran_slam': [18, 20, 20, 8, 14, 7, 11],  
    'master_1000': [36, 36, 28, 22, 11, 19, 15],  
    'otros': [5, 1, 6, 5, 5, 3, 2]  
}  
labels_filas = ["H01", "H02", "H03", "H04", "H05", "H06", "H07"]
```

```
df = pd.DataFrame(data=datos, index=labels_filas)
```

```
print(df.shape)  
# print(df.columns)  
# print(df['pais'])  
# print(type(df['pais']))  
# print(df['pais'].unique())
```

loc y iloc

En Pandas, loc y iloc son atributos que se utilizan para la indexación y selección de datos en un DataFrame. La diferencia principal entre ellos radica en cómo acceden a los datos en función del tipo de índice que se utiliza.

- **loc:** Se utiliza para acceder a los datos utilizando etiquetas o nombres de fila y columna. Es decir, se accede a los datos utilizando los nombres de las filas y columnas específicas. La sintaxis general para usar loc es `df.loc[filas, columnas]`, donde filas y columnas pueden ser **etiquetas** de índice o listas de etiquetas de índice. Por ejemplo, `df.loc[3, 'Nombre']` accede al valor en la fila con etiqueta 3 y la columna 'Nombre'.
- **iloc:** Se utiliza para acceder a los datos utilizando índices enteros basados en la posición. Es decir, se accede a los datos utilizando las posiciones numéricas de las filas y columnas. La sintaxis general para usar iloc es `df.iloc[filas, columnas]`, donde filas y columnas son **índices enteros** o listas de índices enteros. Por ejemplo, `df.iloc[2, 1]` accede al valor en la tercera fila y la segunda columna.

Ejemplo

```
fila = df.loc["H03"]  
print(fila)
```

```
# slicing  
print(df.iloc[2:4])  
#print(df.loc["H03":"H06"])
```

```
#2 columnas  
df[["tenista","master_1000"]]
```

```
# 1 casilla  
print(df.at["H03","tenista"])  
print(df.loc["H03","tenista"])  
print(df.iloc[2,0])
```

	tenista	pais	gran_slam	master_1000	otros
H01	Novak Djokovic	Serbia	18	36	5
H02	Rafael Nadal	España	20	36	1
H03	Roger Federer	Suiza	20	28	6
H04	Ivan Lendl	República Checa	8	22	5
H05	Pete Sampras	Estados Unidos	14	11	5
H06	John McEnroe	Estados Unidos	7	19	3
H07	Bjorn Borg	Suecia	11	15	2

DataFrame

índice

tenista	Roger Federer
pais	Suiza
gran_slam	20
master_1000	28
otros	6

dataframe.loc["H03"]

Series

Filtrados

```
# Queremos saber qué tenistas ganaron 20 Gran Slam  
print( df[df["gran_slam"] == 20] )
```

```
# Queremos saber qué tenistas ganaron 20 Gran Slam y son de Suiza  
df[(df["gran_slam"] == 20) & (df["pais"] == "Suiza")]
```

Agregar/Eliminar

Crear una columna total con todos los torneos ganados.

```
df['total'] = df["gran_slam"] + df["master_1000"] + df["otros"]
```

```
print(df)
```

```
# df.drop(['total'], axis=1, inplace=True)
```

Archivo CSV

```
import pandas as pd

data_set = pd.read_csv('netflix_titles.csv', encoding='utf-8')

print(data_set.shape)
print(data_set.columns)
print(data_set.head(7))
print(data_set.tail())

print(data_set['type'].unique())
print(data_set['type'].value_counts())
```

groupby()

Queremos saber cuántos títulos hay por tipo y año

```
data_set.groupby(["type", "release_year"])["title"].count()
```

Cuantos años hubieron títulos de tipo movie

```
grupos = data_set.groupby(["type", "release_year"])["title"].count()  
print(grupos["Movie"].count())
```

groupby()

Queremos saber los 10 años con más títulos

```
cantidad_x_anio = data_set.groupby(["release_year"])["title"].count()  
print(cantidad_x_anio)
```

Los 10 años con mas titulos

```
print(cantidad_x_anio.sort_values().tail(10))
```

```
# print(cantidad_por_año.sort_values(ascending=False).head(10))
```

Ejemplo

Mostrar y guardar todos los TV shows de Argentina.

```
# filtramos
```

```
tv_shows_ar = data_set[(data_set["type"] == "TV Show") & (data_set["country"] == "Argentina")]
```

```
# guardamos en un archivo csv
```

```
tv_shows_ar.to_csv("ShowsAR.csv")
```

```
# guardamos en un archivo json
```

```
tv_shows_ar.to_json("ShowsAR.json")
```

Ejercicio 1

Escribir un programa que pregunte al usuario las ganancias por año del 2000 al 2005 y muestre por pantalla dos Series con la cantidad de ganancias indexada por los años, una antes y otra después de aplicarles un descuento del 10%.

Ejercicio 2

Escribir programa que genere y muestre por pantalla un DataFrame con los datos de la siguiente tabla:

Mes	Ventas	Gastos
Enero	30500	22000
Febrero	35600	23400
Marzo	28300	18100
Abril	33900	20700

Ejercicio 3

Escribir una función que reciba un DataFrame con el formato del ejercicio anterior, que agregue al df la columna balance (ventas - gastos) y devuelva su promedio.

hint: `.mean()`

Ejercicio 4

Del data set de netflix:

1. Mostrar las películas de Francia del siglo XX.
2. Informar la cantidad de TV entre los años 1980 y 2000
3. Mostrar el año con más y con menos, películas y tv shows.